CANVASOS

The Open-Source Universal Art Creation Platform

Master Technical Documentation

PRD · FRS · API Reference · Technical Specification · Architecture

Version 1.0 | March 2026

Open Source · MIT License

Table of Contents

PART I — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Vision & Mission

CanvasOS is the world's first open-source, all-in-one creative operating system for artists of every discipline. It unifies digital painting, AI art generation, 3D sculpting, vector design, motion graphics, music production, generative code art, photography manipulation, and narrative writing into a single, fluid, cross-platform workspace.

Mission: Democratize professional-grade creative tools by integrating every major art medium into one beautifully simple, infinitely extensible, community-owned platform.

2. Problem Statement

Today's artists must juggle 8–15 separate applications to work across disciplines. Switching tools breaks creative flow, multiplies licensing costs, and creates incompatible file formats. Open-source alternatives exist per category but none integrate across all categories. CanvasOS solves this by making every tool a module within a single unified canvas.

Pain Point	Current Reality	CanvasOS Solution
Tool fragmentation	8-15 apps per artist	Single unified workspace
Cost barrier	$500–$2000/yr in subscriptions	Free & open source forever
Format incompatibility	Export/import cycles break flow	Native cross-module asset sharing
AI tool silos	Separate accounts per AI service	Unified AI pipeline with local fallback
Collaboration friction	No creative suite built for real-time co-creation	Built-in multiplayer canvas
No programmability	Closed ecosystems resist automation	Full plugin/scripting API

3. Target Users & Personas

Persona 1 — The Multidisciplinary Artist

Age 22–38 | Creates across illustration, motion, and music | Currently uses Procreate + After Effects + Ableton | Wants one tool that respects their workflow without sacrificing quality.

Persona 2 — The AI Art Explorer

Age 18–45 | Rapid prototyping with Midjourney, DALL·E, Stable Diffusion | Wants to combine AI-generated assets with manual refinement, 3D composition, and music scoring in the same canvas.

Persona 3 — The Open-Source Developer-Artist

Age 25–40 | Machine learning background | Builds generative art with code | Wants a platform where TouchDesigner-style visuals, p5.js sketches, and GPU shaders coexist with painting and writing tools.

Persona 4 — The Indie Studio

Team of 2–10 | Creates games, films, or interactive experiences | Needs real-time collaboration, asset versioning, and a single project file that holds every asset type.

4. Goals & Success Metrics

Goal	Key Metric	Year 1 Target
Reach	GitHub stars	50,000+
Adoption	Monthly active creators	100,000+
Engagement	Avg. session length	> 45 minutes
Quality	Module feature parity vs. top alternatives	> 80% per module
Community	Open PRs merged / month	> 200
Performance	Canvas FPS at max zoom	> 60 FPS
AI speed	Image generation latency (cloud)	< 8 seconds
Stability	Crash rate	< 0.1%

5. Core User Stories

The following user stories define the minimum viable feature set for v1.0:

- As an illustrator, I want a pressure-sensitive brush engine with layers, blend modes, and a color wheel so I can paint as naturally as in Procreate.
- As an AI artist, I want to type a prompt and receive multiple AI-generated image variants directly on my canvas so I can iterate rapidly without leaving the app.
- As a 3D sculptor, I want to sculpt meshes in-app and render them to a 2D layer so my 3D work integrates seamlessly with painting.
- As a motion designer, I want a timeline where I can keyframe any element on the canvas and export it as an MP4 or GIF.
- As a musician, I want to compose a score directly inside the app and attach it to an animation timeline so visuals and audio stay in sync.
- As a developer-artist, I want to write live-coded generative art in JavaScript/GLSL and see results in real-time on the canvas.

- As a photographer, I want to import RAW files, apply non-destructive adjustments, and composite them with illustrated layers.
- As a writer, I want a prose editor where I can attach illustrations, so my narrative and visuals are one unified artifact.
- As a collaborator, I want to share a live link so another artist can work on the same canvas simultaneously with cursor presence.
- As any user, I want all my work to auto-save to local storage and optionally sync to a cloud project vault.

6. Feature Prioritization (MoSCoW)

Must Have (v1.0)

- Unified canvas with infinite zoom and layers
- Digital painting engine (brush, pencil, fill, selection tools)
- AI image generation via API (OpenAI, Stability AI, Replicate)
- Vector drawing tools (pen, shape, boolean operations)
- Basic 3D viewport with sculpting and rendering to canvas
- Timeline-based animation with keyframing
- Photo import and non-destructive editing filters
- Project file format (.cvos) that bundles all assets
- Export: PNG, JPEG, SVG, MP4, GIF, PDF
- Cross-platform PWA with offline support

Should Have (v1.1)

- Real-time multiplayer collaboration
- Music/audio composition layer with MIDI support
- Live code editor for generative art (p5.js/GLSL)
- Plugin SDK and marketplace
- AI-assisted tools: inpainting, outpainting, style transfer
- Asset library with tagging, search, and smart collections

Could Have (v1.2+)

- Local AI model running (Stable Diffusion via ONNX/WebGPU)
- Full node-based compositing graph
- AR/VR canvas preview
- Desktop native app (Electron/Tauri) with GPU acceleration
- AI music generation (Suno/Udio API integration)

Won't Have (v1.x)

- Proprietary cloud storage subscription
- Non-modular locked feature tiers
- Any telemetry without explicit opt-in

7. Product Roadmap

Phase	Timeline	Milestone
Foundation	Month 1–2	Core canvas, layers, brush engine, PWA shell, project file format
Drawing Suite	Month 2–3	Full painting toolset, vector tools, photo import
AI Integration	Month 3–4	AI generation APIs, inpaint, outpaint, prompt panel
3D & Animation	Month 4–5	Basic sculpting, timeline, keyframing, GIF/MP4 export
Code & Audio	Month 5–6	Live code editor, audio layer, MIDI playback
Collaboration	Month 6–7	Real-time multiplayer, cloud sync, share links
Plugins & SDK	Month 7–8	Plugin API, community marketplace, scripting console
v1.0 Release	Month 9	Public launch, documentation, community onboarding

PART II — FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

This section defines detailed functional requirements for each module of CanvasOS. Requirements use the identifiers FR-[MODULE]-[NNN] for traceability.

Module 1 — Canvas Engine (CVSE)

ID	Requirement	Priority
FR-CVSE-001	The canvas shall support infinite pan and zoom from 1% to 6400% with smooth GPU-accelerated rendering	Must
FR-CVSE-002	The canvas shall support unlimited layers with groups, masks, and clipping masks	Must
FR-CVSE-003	Layer blend modes shall include: Normal, Multiply, Screen, Overlay, Soft Light, Hard Light, Color Dodge, Color Burn, Difference, Hue, Saturation, Color, Luminosity	Must
FR-CVSE-004	The canvas shall support non-destructive adjustment layers (Hue/Sat, Curves, Levels, Color Balance)	Should
FR-CVSE-005	Undo/redo history shall retain a minimum of 500 steps	Must
FR-CVSE-006	The system shall render at minimum 60 FPS on a 4K canvas with 20 active layers	Must
FR-CVSE-007	Canvas size shall be configurable from 32x32 px to 16384x16384 px at any DPI	Must
FR-CVSE-008	The canvas shall support multi-touch gestures on touch devices (pinch-zoom, two-finger rotate)	Must

Module 2 — Digital Painting Engine (DPNT)

ID	Requirement	Priority
FR-DPNT-001	The brush engine shall simulate pressure sensitivity via stylus API and mouse pressure emulation	Must
FR-DPNT-002	Brush types shall include: hard round, soft round, textured, ink, watercolor, airbrush, smear, clone stamp	Must

ID	Requirement	Priority
FR-DPNT-003	Users shall be able to create, save, and share custom brushes with configurable spacing, flow, opacity, texture, and dynamics	Must
FR-DPNT-004	The color system shall provide: color wheel, HSB/RGB/CMYK/HEX input, swatches palette, eyedropper, gradient editor	Must
FR-DPNT-005	Selection tools shall include: rectangle, ellipse, lasso, magic wand, quick mask, select-by-color	Must
FR-DPNT-006	Transform operations shall include: scale, rotate, skew, warp (mesh warp), perspective transform	Should
FR-DPNT-007	The fill tool shall support solid, gradient, and pattern fills with flood and paint bucket modes	Must
FR-DPNT-008	Brushes shall support texture stamping from user-imported images	Should

Module 3 — AI Art Generation (AIGEN)

ID	Requirement	Priority
FR-AIGEN-001	The AI panel shall support text-to-image generation via OpenAI DALL·E 3, Stability AI (SDXL), and Replicate APIs	Must
FR-AIGEN-002	Users shall be able to configure API keys per provider with secure local storage (never transmitted to CanvasOS servers)	Must
FR-AIGEN-003	AI outputs shall be inserted directly as canvas layers with full editability	Must
FR-AIGEN-004	The system shall support inpainting (painting over a masked region and regenerating with AI)	Should
FR-AIGEN-005	The system shall support outpainting (expanding the canvas boundary with AI generation)	Should
FR-AIGEN-006	Image-to-image generation (img2img) shall be supported with adjustable strength parameter	Should
FR-AIGEN-007	A prompt history panel shall retain the last 200 prompts per project	Must
FR-AIGEN-008	Batch generation shall allow generating 1–8 variants simultaneously with the same prompt	Should

ID	Requirement	Priority
FR-AIGEN-009	The system shall support local AI inference via WebGPU (Stable Diffusion ONNX) as an offline fallback	Could
FR-AIGEN-010	Style reference: users shall be able to drag an existing image to influence the AI generation style	Should

Module 4 — 3D Sculpting & Rendering (3DSR)

ID	Requirement	Priority
FR-3DSR-001	A 3D viewport shall be embedded within the canvas as a specialized layer type	Must
FR-3DSR-002	Sculpting tools shall include: draw, smooth, flatten, pinch, crease, clay strips, snake hook	Must
FR-3DSR-003	Primitive shapes (sphere, cube, cylinder, torus, plane) shall be available as starting meshes	Must
FR-3DSR-004	The renderer shall support PBR materials with albedo, roughness, metallic, and normal map inputs	Should
FR-3DSR-005	Users shall be able to render the 3D scene to a 2D canvas layer at configurable resolution	Must
FR-3DSR-006	Import formats: OBJ, GLTF/GLB, STL, PLY	Should
FR-3DSR-007	Export formats: OBJ, GLTF/GLB, STL	Should
FR-3DSR-008	AI text-to-3D generation via Meshy AI or Tripo3D API shall be supported	Could
FR-3DSR-009	Basic rigging and pose tools for character meshes	Could

Module 5 — Vector Design (VECT)

ID	Requirement	Priority
FR-VECT-001	The vector engine shall use SVG-based path rendering with Bezier curve editing (pen tool)	Must
FR-VECT-002	Shape tools: rectangle, ellipse, polygon, star, line, arrow	Must
FR-VECT-003	Boolean operations: Union, Subtract, Intersect, Exclude, Divide	Must

ID	Requirement	Priority
FR-VECT-004	Typography: full font rendering with kern/tracking/leading, OpenType features	Must
FR-VECT-005	Pathfinder/shape builder for complex compound paths	Should
FR-VECT-006	SVG import and export with full path fidelity	Must
FR-VECT-007	Vector and raster layers shall be compositable in the same canvas	Must
FR-VECT-008	Grid, snap-to-grid, snap-to-path, and alignment guides	Must

Module 6 — Animation & Motion (ANIM)

ID	Requirement	Priority
FR-ANIM-001	A non-linear timeline shall support keyframing of position, scale, rotation, opacity, and filter properties for any layer	Must
FR-ANIM-002	Easing curves (linear, ease-in, ease-out, ease-in-out, spring, custom bezier) shall be configurable per keyframe	Must
FR-ANIM-003	Frame-by-frame animation shall be supported with onion skinning	Must
FR-ANIM-004	Export formats: MP4 (H.264/H.265), GIF, WebM, APNG, image sequence (PNG/JPEG)	Must
FR-ANIM-005	The animation engine shall playback at minimum 60 FPS for compositions up to 1080p	Must
FR-ANIM-006	Pre-built motion presets: fade, slide, pop, shake, bounce	Should
FR-ANIM-007	3D layer transformations (depth, camera perspective) in the timeline	Could
FR-ANIM-008	Audio waveform visualization shall be displayed in the timeline	Should

Module 7 — Music & Audio (MAUD)

ID	Requirement	Priority
FR-MAUD-001	An audio timeline layer shall support importing WAV, MP3, FLAC, OGG, AAC files	Must

ID	Requirement	Priority
FR-MAUD-002	A basic step sequencer shall allow composing beats and melodies within the app	Should
FR-MAUD-003	A synth engine (Tone.js-based) shall provide: oscillator, filter, envelope, reverb, delay	Should
FR-MAUD-004	MIDI input from connected devices shall be supported via Web MIDI API	Should
FR-MAUD-005	AI music generation via Suno or Udio API with prompt-to-audio shall be supported	Could
FR-MAUD-006	Non-destructive audio effects: EQ, compressor, reverb, delay, pitch shift, time stretch	Should
FR-MAUD-007	Audio shall be synchronized to the animation timeline frame-accurately	Must
FR-MAUD-008	Export: WAV (master), MP3, FLAC, embedded in MP4	Must

Module 8 — Generative & Code Art (CGEN)

ID	Requirement	Priority
FR-CGEN-001	An embedded code editor (Monaco Editor) shall support JavaScript, GLSL, and p5.js	Should
FR-CGEN-002	Live preview of generative code shall update in real-time as the user types	Should
FR-CGEN-003	Code layers shall render their output to a canvas layer as an image or animation	Should
FR-CGEN-004	Users shall be able to capture a code layer output as a static frame to a raster layer	Should
FR-CGEN-005	A library of example generative art sketches shall be bundled (fractals, noise fields, particle systems)	Could
FR-CGEN-006	WebGL/WebGPU shaders shall be executable as real-time animated layers	Could
FR-CGEN-007	The code environment shall expose a CanvasOS Canvas API for reading/writing pixel data of sibling layers	Could

Module 9 — Photography (PHOT)

ID	Requirement	Priority
FR-PHOT-001	Import support: JPEG, PNG, TIFF, WEBP, HEIC, RAW (Canon CR2/CR3, Nikon NEF, Sony ARW, Adobe DNG)	Must
FR-PHOT-002	Non-destructive editing: Exposure, Contrast, Highlights, Shadows, Whites, Blacks, Clarity, Vibrance, Saturation	Must
FR-PHOT-003	Color grading: HSL per-channel, Tone Curve, Split Toning, Color Mixer	Should
FR-PHOT-004	Healing brush, clone stamp, spot removal tools for photo retouching	Should
FR-PHOT-005	AI-powered background removal (remove.bg API or local segmentation model)	Should
FR-PHOT-006	Lens correction: distortion, chromatic aberration, vignette	Should
FR-PHOT-007	AI Super-Resolution upscaling (Real-ESRGAN or similar)	Could
FR-PHOT-008	Batch photo editing via macro recording and playback	Could

Module 10 — Writing & Narrative (WRIT)

ID	Requirement	Priority
FR-WRIT-001	A rich text editor layer shall support prose, poetry, and structured writing within the canvas	Should
FR-WRIT-002	Text layers shall be linkable to illustration layers to create illustrated narratives or comics	Should
FR-WRIT-003	An AI writing assistant (Claude / GPT-4) shall be available for prose suggestions, continuations, and edits	Should
FR-WRIT-004	Export: EPUB, PDF, MD, DOCX for written works	Should
FR-WRIT-005	A comic/storyboard layout mode shall provide panel grids that hold canvas layers + text	Could
FR-WRIT-006	Twine-compatible interactive fiction export (JSON format)	Could

Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	First contentful paint < 2s on 4G; canvas operations < 16ms (60 FPS)
NFR-002	Accessibility	WCAG 2.1 AA compliance; full keyboard navigation; screen reader support
NFR-003	Localisation	UI strings externalized; support for RTL layouts; initial languages: EN, ES, FR, ZH, AR
NFR-004	Security	API keys stored in OS keychain / encrypted local storage; no plaintext secrets in logs
NFR-005	Privacy	No analytics without opt-in; no user data sent to CanvasOS servers by default
NFR-006	Offline	Core editing features functional offline (PWA service worker caching); AI features gracefully degrade
NFR-007	Extensibility	Plugin API must be stable across minor versions; breaking changes require major version bump
NFR-008	Compatibility	Support: Chrome 120+, Firefox 120+, Safari 17+, Edge 120+; iOS 16+; Android 10+
NFR-009	File Size	Core app bundle < 5 MB (gzipped); lazy-load all modules
NFR-010	Licensing	All bundled dependencies must be MIT, Apache 2.0, BSD, or CC0 compatible

PART III — TECHNICAL SPECIFICATION

1. Architecture Overview

CanvasOS follows a modular monorepo architecture with a shared core rendering engine and independently loadable feature modules. The system is designed for progressive enhancement: every feature that can run locally runs locally, and cloud/AI services are opt-in.

High-Level Architecture

Layer	Responsibility	Technology
Presentation	UI components, panels, toolbars, dialogs	React 18 + TypeScript
Canvas Renderer	GPU-accelerated pixel and vector rendering	WebGL2 / WebGPU (via PixiJS + custom shaders)
Module Layer	Feature modules (paint, AI, 3D, audio, etc.)	Dynamically imported ES modules
State Management	Global app state, undo/redo, history	Zustand + Immer
AI Gateway	Routes AI requests to configured providers	Node.js microservice (optional)
Storage	Local project files, cloud sync, asset library	IndexedDB + OPFS + S3-compatible
Collaboration	Real-time multi-user canvas sync	Yjs (CRDT) + WebSocket
Plugin Host	Sandboxed execution of third-party plugins	Web Workers + Comlink

2. Tech Stack

Frontend

- Framework: React 18 with TypeScript 5
- Build: Vite 5 with module federation for lazy-loading art modules
- State: Zustand for global state; Jotai for per-module atom state
- Canvas: PixiJS 8 (WebGL2) for raster; Paper.js for vector; Three.js r165 for 3D
- Animation: GSAP (motion engine), Lottie (playback), custom keyframe engine
- Audio: Tone.js 15 (synth/sequencer); Web Audio API (raw audio); Web MIDI API
- Code Editor: Monaco Editor (VS Code engine)
- Collaboration: Yjs with y-websocket provider
- Styling: Tailwind CSS v4 with design tokens
- PWA: Workbox 7 (service worker, offline caching)

- Desktop: Tauri 2 wrapper (Rust backend, <10MB overhead)
- Mobile: Capacitor 6 for iOS/Android packaging

Backend (Optional — Self-Hostable)

- Runtime: Node.js 22 LTS with Fastify 5 framework
- AI Gateway: Python 3.12 + FastAPI (handles Stable Diffusion local inference)
- Database: PostgreSQL 16 (projects, users, assets metadata)
- Cache: Redis 7 (session, rate-limiting, AI job queue)
- Queue: BullMQ (async AI generation jobs)
- Object Storage: MinIO (S3-compatible, self-hosted) or any S3 provider
- Realtime: WebSocket via Fastify WS plugin
- Auth: Lucia v3 (sessions) with OAuth2 (GitHub, Google)

Infrastructure (Default Recommended)

- Containerization: Docker + Docker Compose (all services in one compose file)
- CI/CD: GitHub Actions (test, lint, build, deploy)
- Hosting: Vercel Edge (frontend) + Fly.io (backend) — both free tiers sufficient for small teams
- CDN: Cloudflare (assets, model weights)
- Monitoring: OpenTelemetry + Grafana (optional, self-hosted)

3. Project File Format (.cvos)

A .cvos file is a ZIP archive with the following structure. All metadata is stored as UTF-8 JSON. Binary assets use their native format to avoid re-encoding losses.

```
canvasos-project.cvos
├── manifest.json          # Project metadata, module versions, canvas config
├── canvas.json            # Layer tree, blend modes, visibility, transform data
├── timeline.json          # Keyframes, markers, audio sync data
├── assets/                # Raw asset files (PNG, SVG, OBJ, MP3, etc.)
│   ├── layers/            # Raster layer data (per-layer PNG snapshots)
│   ├── media/             # Audio, video files
│   └── 3d/                # 3D mesh files
├── code/                  # Code art scripts (.js, .glsl)
├── plugins/               # Plugin state and config snapshots
└── preview.png            # 512x512 project thumbnail
```

4. Canvas Rendering Pipeline

The rendering pipeline follows a layer compositor pattern with a GPU-resident texture cache. All operations are non-destructive until the user explicitly flattens layers.

Stage	Description	Technology
Input	Stylus/mouse/touch events captured at native device rate	Pointer Events API

Stage	Description	Technology
Stroke Path	Pressure-sensitive bezier path construction with predictive smoothing	Custom TypeScript engine
GPU Upload	Stroke data uploaded to WebGL texture atlas per layer	PixiJS RenderTexture
Compositor	Layer blend modes applied via custom GLSL fragment shaders	WebGL2 GLSL
Viewport	Final composited frame output to canvas element at display Hz	requestAnimationFrame
Thumbnail	Background thread generates layer and project thumbnails	OffscreenCanvas Worker

5. AI Integration Architecture

CanvasOS implements a provider-agnostic AI interface. All AI calls flow through the ArtGenClient class, which selects the provider based on user configuration.

```
interface ArtGenProvider {
  id: string;                                          // e.g. 'openai' |
'stability' | 'replicate'
  textToImage(params: T2IParams): Promise<Blob[]>;
  imageToImage(params: I2IParams): Promise<Blob[]>;
  inpaint(params: InpaintParams): Promise<Blob[]>;
  outpaint(params: OutpaintParams): Promise<Blob[]>;
}

// Supported providers in v1.0
providers: ['openai-dalle3', 'stability-sdxl', 'stability-sd3',
            'replicate-flux', 'replicate-sdxl', 'local-webgpu']
```

6. Plugin System Architecture

Plugins run in isolated Web Workers and communicate with the main thread via a message-passing API. This prevents plugins from blocking the canvas and sandboxes their access to the filesystem and network.

```
// Plugin manifest (plugin.json)
{
  "id": "my-plugin",
  "name": "My Plugin",
  "version": "1.0.0",
  "entry": "index.js",
  "permissions": ["canvas.read", "canvas.write", "network.fetch"],
  "ui": "panel.html"
}

// Plugin API (available inside Web Worker)
```

```
canvasOS.canvas.getLayers(): Promise<Layer[]>
canvasOS.canvas.insertLayer(layer: LayerInit): Promise<string>
canvasOS.canvas.getPixels(layerId, rect): Promise<ImageData>
canvasOS.canvas.putPixels(layerId, imageData, rect): Promise<void>
canvasOS.ui.showPanel(html: string): void
canvasOS.storage.get(key: string): Promise<unknown>
canvasOS.storage.set(key: string, value: unknown): Promise<void>
```

7. Database Schema (PostgreSQL)

The following tables are required for the optional cloud backend. All fields are nullable unless marked NOT NULL.

Table	Key Columns	Purpose
users	id (UUID), email NOT NULL, display_name, avatar_url, created_at	User accounts
projects	id (UUID), owner_id FK, title NOT NULL, thumbnail_url, is_public, updated_at	Project metadata
project_versions	id, project_id FK, version_num, file_size, storage_key, created_at	Version history
assets	id, owner_id FK, project_id FK, name, mime_type, storage_key, width, height, duration	Media asset catalog
collaborators	project_id FK, user_id FK, role (viewer/editor/owner), invited_at	Project access control
ai_jobs	id, user_id FK, provider, model, prompt, status, result_urls, tokens_used, created_at	AI generation audit
plugins	id, name, version, author_id FK, manifest JSONB, download_count, is_verified	Plugin registry
sessions	id, user_id FK, token_hash, expires_at, ip, user_agent	Auth sessions

8. Security Model

- API keys (OpenAI, Stability AI, etc.) are encrypted at rest using AES-256-GCM with a key derived from a user passphrase via PBKDF2. They never leave the local device unless the user explicitly enables cloud key sync.
- All backend API endpoints require a valid session token (HttpOnly cookie). CSRF protection via SameSite=Strict cookies and custom headers.
- Plugin permissions are enforced at the Web Worker boundary via a capability-based access control list. Plugins cannot read/write outside their declared permission scope.
- File uploads are validated for MIME type, magic bytes, and size (max 500 MB per file) before storage.
- Rate limiting: AI generation endpoints are rate-limited per user (60 requests/hour default, configurable by self-hosters).

- Dependency security: automated Dependabot alerts; all PRs run npm audit before merge.

PART IV — API REFERENCE

The CanvasOS REST API powers the optional cloud backend and plugin integrations. All API endpoints are prefixed with /api/v1. Responses use JSON. Authentication uses Bearer tokens (JWTs) or session cookies.

Authentication

POST /api/v1/auth/register

Field	Type	Required	Description
email	string	Yes	User email address
password	string	Yes	Min 8 characters
display_name	string	No	Public display name

POST /api/v1/auth/login

Returns a session token. Store as HttpOnly cookie or in-memory. Never store in localStorage.

POST /api/v1/auth/logout

Invalidates the current session token.

GET /api/v1/auth/me

Returns the authenticated user object.

Projects API

Method	Endpoint	Description
GET	/projects	List all projects for the authenticated user
POST	/projects	Create a new project (returns project ID + upload URL)
GET	/projects/:id	Get project metadata
PATCH	/projects/:id	Update project title, visibility, or thumbnail
DELETE	/projects/:id	Soft-delete a project (recoverable for 30 days)
GET	/projects/:id/versions	List all saved versions of a project

Method	Endpoint	Description
POST	/projects/:id/versions	Save a new version (multipart upload of .cvos file)
GET	/projects/:id/versions/:versionId/download	Download a specific version
POST	/projects/:id/fork	Fork a public project to the authenticated user's workspace
GET	/projects/:id/collaborators	List collaborators and their roles
POST	/projects/:id/collaborators	Invite a collaborator by email
DELETE	/projects/:id/collaborators/:userId	Remove a collaborator

AI Generation API

Method	Endpoint	Description
POST	/ai/generate	Text-to-image generation (provider-agnostic)
POST	/ai/inpaint	Inpainting with masked image input
POST	/ai/outpaint	Outpainting to expand canvas boundaries
POST	/ai/img2img	Image-to-image transformation
GET	/ai/jobs/:jobId	Poll async generation job status
GET	/ai/jobs/:jobId/result	Download generation result when complete
GET	/ai/history	Get the user's AI generation history
POST	/ai/music/generate	AI music generation (Suno/Udio integration)

POST /api/v1/ai/generate — Request Body

Field	Type	Required	Default	Description
prompt	string	Yes	—	Text description of the desired image

Field	Type	Required	Default	Description
provider	string	No	openai	openai | stability | replicate | local
model	string	No	auto	Provider-specific model ID
width	integer	No	1024	Output image width in pixels (64–2048)
height	integer	No	1024	Output image height in pixels (64–2048)
n	integer	No	1	Number of variants (1–8)
negative_prompt	string	No	—	What to avoid in the generation
style_reference_url	string	No	—	URL of a reference image for style transfer
strength	float	No	0.8	Img2img denoising strength (0.0–1.0)
seed	integer	No	random	Random seed for reproducible outputs

Assets API

Method	Endpoint	Description
GET	/assets	List all assets with pagination and filter
POST	/assets/upload-url	Request a pre-signed S3 upload URL for a file
POST	/assets	Register an uploaded asset into the asset library
GET	/assets/:id	Get asset metadata and download URL
DELETE	/assets/:id	Delete an asset
GET	/assets/search?q=:query	Full-text search across asset names and tags
POST	/assets/:id/tags	Add tags to an asset
DELETE	/assets/:id/tags/:tag	Remove a tag from an asset

Plugin Registry API

Method	Endpoint	Description
GET	/plugins	Browse published plugins (paginated, filterable by category)
GET	/plugins/:id	Get plugin details, manifest, and download link
POST	/plugins	Submit a new plugin (requires author account)
PATCH	/plugins/:id	Update plugin metadata or publish new version
POST	/plugins/:id/install	Record a plugin install event (analytics)
GET	/plugins/installed	Get list of plugins installed in current workspace

WebSocket Events (Collaboration)

Real-time collaboration uses Yjs over WebSocket. The following custom events are broadcast on top of the Yjs sync protocol:

Event	Direction	Payload	Description
cursor.move	Client → All	{userId, x, y, color}	Broadcast cursor position to collaborators
layer.lock	Client → All	{userId, layerId}	Notify others a layer is being edited
layer.unlock	Client → All	{userId, layerId}	Release a layer lock
ai.generating	Client → All	{userId, jobId, prompt}	Show AI generation status to team
ai.complete	Server → All	{jobId, layerId}	Notify team when AI result is placed
user.join	Server → All	{userId, displayName, color}	User joined the session
user.leave	Server → All	{userId}	User left the session
chat.message	Client → All	{userId, text, timestamp}	In-canvas chat message

PART V — AGENT BUILD PROMPT

The following prompt is designed to be given verbatim to an AI coding agent (e.g. Claude Code, Cursor, Devin, GPT-Engineer) to scaffold and build CanvasOS. It is self-contained and references the specifications in Parts I–IV of this document.

AGENT TASK: BUILD CANVASOS — OPEN-SOURCE UNIVERSAL ART PLATFORM

You are a senior full-stack developer. Your mission is to scaffold and build CanvasOS, an open-source, cross-platform art creation platform that integrates digital painting, AI generation, 3D sculpting, vector design, animation, music, generative code art, photography, and writing into a single unified canvas.

PHASE 0 — Repository Setup

1. Initialize a Turborepo monorepo with the following packages: apps/web (Vite + React + TypeScript), apps/desktop (Tauri 2 wrapping apps/web), apps/mobile (Capacitor 6 wrapping apps/web), packages/core (canvas engine, state, file format), packages/ui (shared design system components), packages/ai (AI provider abstractions), packages/modules/paint, packages/modules/vector, packages/modules/sculpt-3d, packages/modules/animate, packages/modules/audio, packages/modules/codeart, packages/modules/photo, packages/modules/write, services/api (Fastify backend), services/ai-gateway (Python FastAPI for local inference).
2. Configure TypeScript 5 strict mode, ESLint (airbnb-typescript preset), Prettier, Husky pre-commit hooks, and Vitest for unit testing across all packages.
3. Create the root README.md with: project description, architecture diagram (as ASCII), quickstart instructions (npx canvasos or docker compose up), contribution guide, and MIT License.

PHASE 1 — Core Canvas Engine (packages/core)

4. Implement CanvasState using Zustand with Immer. State shape must include: layers (tree structure with id, name, type, visible, locked, opacity, blendMode, transform), history (CircularBuffer of 500 snapshots for undo/redo), viewport (zoom, panX, panY), selection (active tool, selected layer IDs), project (metadata, .cvos file path).
5. Build the LayerCompositor class using PixiJS 8 RenderTextures. Each layer is a WebGL texture. Implement all 18 blend modes as GLSL fragment shaders (do not use CSS mix-blend-mode — it is too slow for canvas use). The compositor must render the full layer stack at 60+ FPS on a 4K canvas.
6. Implement the .cvos file format reader/writer. Use JSZip for ZIP handling. The manifest.json must include canvasOSVersion, canvasWidth, canvasHeight, dpi, createdAt, updatedAt, and a moduleVersions map. Layer pixel data is stored as LZ4-compressed PNG in assets/layers/.

7. Build a PointerEventHandler that normalizes mouse, touch, and stylus pointer events into a unified StrokePoint { x, y, pressure, tiltX, tiltY, timestamp } stream. Implement stroke smoothing using Catmull-Rom spline interpolation with a configurable latency/quality tradeoff setting.
8. Implement the HistoryManager with branching undo — each action is an Immer patch pair (forward/backward). undo() and redo() apply patches without re-rendering from scratch.

PHASE 2 — Painting Module (packages/modules/paint)

9. Implement BrushEngine with the following built-in brush types: HardRound, SoftRound, Textured (stamp-based), InkPen, Watercolor (wet-edge simulation), Airbrush, Eraser. Each brush uses a stamp-based rendering loop: compute stamp positions along the stroke path, upload stamps to a WebGL brush texture atlas, render stamps to the active layer RenderTexture using additive or normal blending.
10. Build a BrushPresetManager with load/save/export to JSON. Expose brush parameters: size (1–2000px), opacity (0–100%), flow (0–100%), hardness (0–100%), spacing (1–200%), angleDynamics (fixed/direction/random), sizePressure (boolean), opacityPressure (boolean), texturePath (optional asset key).
11. Implement the ColorSystem: HSB/RGB/CMYK/HEX color models with lossless conversion between all models. Build a ColorWheel component (HSB disc), a SwatchPalette (custom + built-in palettes), a GradientEditor (linear/radial/conic), and a PipetteEyedropper (reads from the composited output, not individual layers).
12. Implement selection tools: RectangularMarquee, EllipticalMarquee, LassoTool (freehand polygon), MagicWand (flood-fill based on color similarity threshold), QuickMask (paint the selection as a red overlay). Selections must produce a floating mask channel (Uint8ClampedArray) that the brush engine respects.

PHASE 3 — AI Module (packages/ai)

13. Build ArtGenClient as a provider-agnostic TypeScript class. Constructor accepts a ProviderConfig array. The generate() method accepts a GenerateRequest and routes to the appropriate provider. Implement these providers: OpenAIProvider (DALL·E 3 API), StabilityProvider (Stable Diffusion 3 API), ReplicateProvider (FLUX model via Replicate API), LocalProvider (WebGPU inference stub — log a TODO for Phase 7).
14. Build the AI Panel UI component: a left-side docked panel with a prompt textarea, model selector dropdown, width/height sliders, n-variants stepper, negative prompt textarea, seed input, style reference image drag-drop zone. On generate, show a progress overlay on the canvas with skeleton placeholders for each variant. On complete, insert each variant as a new layer.
15. Implement inpainting: when the user makes a selection and clicks 'Generate in Selection', capture the current canvas content inside the selection as the base image, the selection mask as the mask image, and send to the provider's inpaint endpoint. Composite the result back into the selection area with optional feathering.
16. Store prompt history in the project state. Build a PromptHistory sidebar that shows the last 200 prompts with their thumbnail results. Clicking a history item repopulates the prompt panel.

PHASE 4 — Vector Module (packages/modules/vector)

17. Implement the VectorLayer type using paper.js as the vector engine. A VectorLayer holds a paper.Project and renders it to a WebGL texture via an OffscreenCanvas at the

end of each edit operation. This ensures vector content composites correctly with raster layers.

18. Build the PenTool: click to add anchor points, click-drag to create Bezier handles, double-click to close paths. Implement node editing mode (move, add, delete anchor points; convert between smooth and corner nodes). Paths must have configurable stroke (color, width, dash pattern, cap, join) and fill (solid, gradient, pattern, none).
19. Build Shape Primitives: Rectangle, RoundedRectangle, Ellipse/Circle, Polygon (n sides), Star (n points), Line, Arrow. Each shape is a parametric vector object editable after creation.
20. Implement Boolean Operations (Union, Subtract, Intersect, Exclude, Divide) between any two selected vector paths using paper.js PathItem.unite/subtract/intersect/exclude.
21. Build the TextTool: click to place a text frame, type, select fonts from the system font list + bundled Google Fonts (Open Sans, Lato, Playfair Display, Roboto Mono), configure size/tracking/leading/alignment. Text must be editable at any time without rasterizing.

PHASE 5 — Animation Module (packages/modules/animate)

22. Build the Timeline component: a horizontal scrollable track view where each canvas layer has a corresponding track. Tracks contain keyframe diamonds draggable along the time axis. The playhead is a vertical red line the user can scrub. Implement zoom in/out of the timeline (1x–100x time scale).
23. Implement KeyframeEngine: for each animated property (x, y, scaleX, scaleY, rotation, opacity, filterValue), store keyframes as { time: number, value: number, easing: EasingFn }. At render time, interpolate between keyframes using the configured easing function. Implement: linear, easeIn (quadratic), easeOut (quadratic), easeInOut (cubic), spring (critically damped), customBezier (user-defined P1/P2 control points).
24. Implement frame-by-frame animation: each layer can have multiple Frame entries (like an animation cel stack). Build an onion-skinning renderer that shows up to 5 previous and 5 next frames as ghost overlays with configurable opacity.
25. Implement the VideoExporter using the MediaRecorder API (WebM in-browser) and an FFMPEG.wasm post-processor for MP4/GIF/APNG output. Expose quality, framerate (12/24/30/60 FPS), and resolution settings.

PHASE 6 — Audio Module (packages/modules/audio)

26. Build the AudioLayer type that holds a reference to an imported audio file or a Tone.js Transport composition. The audio layer renders as a waveform visualization track in the Timeline.
27. Implement the StepSequencer UI: a 16-step grid with configurable BPM, swing, and 8 instrument rows (kick, snare, hihat, clap, bass, chord, arp, FX). Each cell triggers a Tone.js Sampler (included free drum kit WAV samples from Freesound CC0). The sequencer drives Tone.Transport which is synchronized to the animation timeline.
28. Build a MiniSynth panel: one Tone.js PolySynth with oscillator type (sine/square/saw/triangle), ADSR envelope, low-pass filter with cutoff and resonance, reverb send, and delay send. Users can play it via MIDI keyboard or the on-screen piano roll.
29. Integrate Web MIDI API: auto-detect connected MIDI devices on load, route MIDI note events to the active synth or sequencer row, route MIDI CC events to configurable parameter mappings (e.g., CC 1 → filter cutoff).

PHASE 7 — Photography Module (packages/modules/photo)

30. Implement PhotoLayer: a non-destructive layer type that wraps an imported raster image. All adjustments are stored as a parameter object and applied at render time via WebGL shaders — never baked into the source pixels. Parameters: exposure, contrast, highlights, shadows, whites, blacks, clarity (local contrast), vibrance, saturation, temperature, tint.
31. Build a Curves editor: 4-channel (RGB composite + R + G + B) interactive curve editor using a Canvas 2D spline UI. Map the curve to a 256-entry lookup table uploaded as a 1D WebGL texture applied during compositing.
32. Implement the background removal tool using the Transformers.js RMBG-1.4 model running in a Web Worker (no external API call required). The output is an alpha mask channel applied to the PhotoLayer. Process at up to 1024x1024; tile larger images.
33. Implement RAW file decoding using libraw.js (WASM build). Support CR2, CR3, NEF, ARW, DNG. Decode to a linear 16-bit RGB buffer, apply white balance and demosaicing, then hand off to the standard PhotoLayer pipeline.

PHASE 8 — Generative Code Module (packages/modules/codeart)

34. Embed Monaco Editor in a CodeLayer panel. Support JavaScript (with type-checked CanvasOS API stub) and GLSL (with syntax highlighting). The editor should hot-reload the sketch output with a 300ms debounce after the user stops typing.
35. For JavaScript sketches: execute user code in an isolated iframe sandbox with postMessage communication. Inject a p5.js instance and a canvasOS object exposing: canvasOS.width, canvasOS.height, canvasOS.frame, canvasOS.captureFrame() (sends current sketch pixels to the canvas layer via postMessage).
36. For GLSL shaders: compile and run as a WebGL fragment shader on a fullscreen quad inside the CodeLayer's RenderTexture. Provide built-in uniforms: u_time (seconds), u_resolution (vec2), u_mouse (vec2), u_texture (sampler2D of the layer below).

PHASE 9 — Collaboration (services/api + packages/core)

37. Set up the Yjs collaborative document structure: the Y.Doc root contains a Y.Map for canvas state. Layer positions, transforms, and visibility are stored as Y.Maps. Brush strokes are appended to per-layer Y.Arrays as compact encoded objects.
38. Implement the y-websocket server in services/api using the Fastify WebSocket plugin. Rooms are keyed by project ID. Support up to 20 concurrent collaborators per document. Persist the Yjs document snapshot to PostgreSQL every 30 seconds.
39. Build the CollaborationPanel UI: a sidebar showing connected users with avatars and colored cursor indicators. Implement real-time cursor broadcasts as custom Yjs awareness fields. Implement layer locking: when a user starts editing a layer, broadcast a lock event; show a colored lock badge on that layer in other users' layer panels.

PHASE 10 — Plugin SDK (packages/core)

40. Implement the PluginHost class: load plugin bundles (index.js + optional panel.html) into a dedicated Web Worker using importScripts. Expose the CanvasOS Plugin API (canvas.read/write, storage, ui.showPanel, network.fetch — gated by declared permissions). Use Comlink to proxy async calls across the Worker boundary.
41. Write the Plugin SDK documentation (packages/plugin-sdk/README.md) with: getting started, API reference, example plugins (color palette generator, AI prompt enhancer, batch export, social media post sizer). Publish as @canvasos/plugin-sdk on npm.

42. Build the Plugin Marketplace UI inside CanvasOS: a grid of plugin cards fetched from GET /api/v1/plugins with search, category filter, install button, and settings gear for installed plugins.

PHASE 11 — Testing & Quality

43. Write unit tests (Vitest) for: ColorSystem conversions, HistoryManager undo/redo, KeyframeEngine interpolation, CVOS file round-trip, ArtGenClient provider routing, BooleanOperations (union/subtract/intersect), StepSequencer BPM timing.
44. Write integration tests (Playwright) for: canvas paint stroke renders correctly, AI generate inserts layer, export to MP4 produces valid file, collaboration: two users see each other's strokes.
45. Set up GitHub Actions CI: on every PR, run lint + type-check + unit tests + build. On merge to main, run integration tests + build + push Docker images to GitHub Container Registry.
46. Implement error boundaries in React for each module panel. All uncaught errors must log to console and show a non-blocking toast (never crash the full app). Include a debug info panel accessible via Cmd/Ctrl+Shift+D.

CRITICAL CONSTRAINTS FOR THE AGENT

- Never use localStorage for sensitive data (API keys). Use the OS keychain via Tauri's keyring plugin for desktop, or AES-256-GCM encrypted IndexedDB for web.
- All AI API keys must be stored client-side only. The CanvasOS backend must never receive, proxy, or log a user's AI provider API keys.
- Every third-party dependency must have a license compatible with MIT. Run license-checker before every release and fail the build if an incompatible license is detected.
- The app must be usable offline. Design every feature with an offline-first mindset. AI features degrade gracefully (show a 'connect to use AI' message). Painting, vector, photo editing, animation, and audio work fully offline.
- The canvas must never block the UI thread. All heavy computation (compression, AI inference, file I/O, audio processing) must run in Web Workers or OffscreenCanvas contexts.
- The default .cvos save must be non-blocking. Auto-save every 60 seconds in a background worker. Never show a blocking 'Saving...' spinner.
- Accessibility first: every tool, panel, and dialog must have keyboard shortcuts, ARIA labels, and focus management that passes axe-core automated accessibility checks.

END OF AGENT PROMPT

PART VI — OPEN SOURCE STRATEGY

Governance Model

CanvasOS uses the Benevolent Dictator For Now (BDFN) model for v0.x releases, transitioning to a community steering committee at v1.0. All architectural decisions are proposed as RFCs (Request for Comments) in GitHub Discussions and require at least 7 days of community review before merging.

Contribution Tiers

Tier	Role	Requirements
Contributor	Submits PRs, files issues	Signed CLA, 1 merged PR
Module Maintainer	Owns one art module	5+ merged PRs, nominated by core team
Core Team	Architectural decisions, release management	Significant sustained contributions, voted in by module maintainers
Steering Committee	Strategic direction, grants, partnerships	Elected annually from core team at v1.0+

Licensing

CanvasOS core is MIT Licensed. The Plugin SDK is also MIT. Third-party integrations (AI provider clients) are MIT. Bundled assets (sample brushes, drum kit samples, fonts) are CC0 or OFL. Documentation is CC BY 4.0.

Community Channels

- GitHub Discussions: feature requests, RFCs, Q&A
- Discord Server: real-time chat, #art-showcase, #plugins, #dev-general
- Monthly Community Call: recorded and posted to YouTube
- Blog (canvasos.art/blog): release notes, tutorials, contributor spotlights
- Twitter/X + Mastodon: @canvasos

Sustainability Model

CanvasOS is free and open-source forever. Sustainability comes from: (1) GitHub Sponsors + Open Collective for individual and corporate donations; (2) a hosted cloud offering (canvasos.cloud) for teams that want managed collaboration, storage, and CI/CD — 100% of this code is also open source; (3) optional verified plugin badges and featured placement in the marketplace for plugin developers.